

Sistema distribuidos

Practica 2

Carlos Francisco Campos Lezameta 50383440P
Jorge Manuel Aquino Sanchez 55336161R
Año 2025/2026

1. Introducción

El proyecto consiste en la simulación de una red de puntos de recarga de vehículos eléctricos, gestionada por una central de control que supervisa, autoriza y monitoriza el funcionamiento de cada estación, así como las solicitudes de recarga enviadas por los conductores a través de sus aplicaciones. La comunicación entre los distintos módulos (Central, Charging Points y Drivers) se realiza mediante sockets TCP y un sistema de mensajería basado en Kafka, garantizando un intercambio de datos en tiempo real y de manera asíncrona.

Esta práctica tiene como objetivo aplicar los conceptos sobre comunicación mediante sockets, streaming de eventos, colas de mensajería y modularidad en el desarrollo de un sistema distribuido realista.

En esta segunda practica hemos expandido y diseñado que doten a nuestro proyecto con capacidades del mundo real como:

- Interoperabilidad mediante APIs REST: Consumo de servicios de terceros (como la plataforma meteorológica OpenWeather) para la toma de decisiones en tiempo real (módulo EV_W), y la exposición de interfaces propias (API_Central y EV_Registry) para permitir la comunicación síncrona entre el Front, el sistema central y los puntos de recarga (CPs).
- Seguridad Extremo a Extremo: Implementación de canales seguros mediante cifrado asimétrico/simétrico para el intercambio de credenciales, mitigando vulnerabilidades frente a ataques de interceptación de datos.
- Trazabilidad y Auditoría (SIEM): Incorporación de un registro estructurado de eventos en el núcleo del sistema (CENTRAL) que permita auditar accesos, fallos de autenticación e incidencias operativas, identificando actores mediante direcciones IP y marcas de tiempo precisas.

2. Implementacion del cifrado.

Para proteger la comunicación de nuestra red de carga y evitar que un atacante intercepte o manipule los datos (ataques Man-In-The-Middle), hemos implementado un sistema de cifrado simétrico de extremo a extremo.

Para ello, utilizamos la librería Fernet (de la suite cryptography de Python), que aplica un cifrado basado en el estándar AES. Esto significa que tanto el emisor como el receptor comparten una única clave secreta por cada Punto de Carga (CP): el emisor cifra el mensaje con ella y el receptor lo descifra.

A continuación, se detalla dónde está este cifrado en el código y cómo funciona en cada uno de tus componentes:

- En el Monitor (EV_CP_M.py)

En la función auxiliar send() y en el proceso de conexión de la clase Monitor.

```
def send(msg, client_socket, fernet):
    if fernet is not None:
        message = fernet.encrypt(msg.encode(FORMAT))
        # Si nos pasan None, lo enviamos en texto claro normal
    else:
        message = msg.encode(FORMAT)

    msg_length = len(message)
    send_length = str(msg_length).encode(FORMAT)
    send_length += b' ' * (HEADER - len(send_length))
    client_socket.send(send_length)
    client_socket.send(message)
```

El Monitor tiene una función send(msg, client_socket, fernet) que intercepta los mensajes. Si la variable fernet tiene una clave cargada, cifra el texto antes de mandarlo por el socket TCP; si es None, lo manda en texto plano.

```
print(f"[DEBUG] Token a enviar: {self.token}")
# 2. Enviar autenticación en claro
msg_auth = f"monitor|AUTENTICACION|{self.ID}|{self.LOC}|{self.token}"
send(msg_auth, self.sock, self.fernet)

respuesta = self.sock.recv(1024).decode(FORMAT)
parts = respuesta.split("|")

if parts[0] == "central" and parts[1] == "OK":
    if len(parts) > 2:
        clave_str = parts[2]
        self.clave_aes = clave_str # ¡AÑADIDO! Nos guardamos la clave

    print(f"[MONITOR] Clave AES recibida: {clave_str}")
    self.fernet = Fernet(clave_str.encode() if isinstance(clave_str, str) else clave_str)
    print("[MONITOR] Cifrado con Fernet configurado exitosamente.")
    return True
```

Al arrancar, el Monitor hace el primer paso en claro para identificarse ante la Central. Tras validar su token JWT, la Central le envía su clave AES definitiva. En ese momento, el Monitor activa el cifrado (`self.fernet = Fernet(clave))` y a partir de ahí, todos sus reportes de estado (`monitor|ESTADO|...`) viajan 100% encriptados.

- En el Engine (`EV_CP_E.py`)
En la función `set_aes_key()` y directamente en la configuración del Productor y Consumidor de Apache Kafka.

```
def consumidor(self):
    topic = f"central-to-engine-{self.id}"
    self.consumer = KafkaConsumer(
        topic,
        bootstrap_servers=[self.ADDR_SERVER],
        auto_offset_reset='latest',
        enable_auto_commit=True,
        group_id=f'engine_group_{self.id}',
        # Desciframos dinámicamente usando self.fernet
        value_deserializer=lambda v: self.fernet.decrypt(v).decode(FORMAT) if v else None
    )
    print(f"[ENGINE] Suscrito correctamente al topic de la Central: {topic}")
```

El Engine recibe la clave criptográfica de forma local a través del Monitor.

Para que el código sea automático y transparente, el cifrado se configuró directamente dentro de las propiedades de Kafka usando funciones cortas (lambdas):

```
def set_aes_key(self, clave_aes):
    """Configura el motor de cifrado y el Productor Kafka dinámicamente"""
    self.fernet = Fernet(clave_aes.encode() if isinstance(clave_aes, str) else clave_aes)

    self.producer = KafkaProducer(
        bootstrap_servers=[self.ADDR_SERVER],
        # Usamos nuestra instancia dinámica de fernet
        value_serializer=lambda v: self.fernet.encrypt(v.encode(FORMAT))
    )
    print("[ENGINE] Clave AES instalada correctamente. Cifrado activado.")
```

`value_serializer`: Cifra automáticamente cualquier dato que el Engine envíe a la Central por el tópico de Kafka.

`value_deserializer`: Descifra de forma automática cualquier orden (como AUTORIZACION o FIN) que le llegue desde la Central.

- En la Central (`EV_Central.py`)

La Central es el cerebro del sistema y se encarga de descifrar las dos vías de comunicación que recibe:

Por Sockets (Con el Monitor): En la función `handle_client` (Paso 2), la Central recibe los bytes cifrados del Monitor y los descifra. Si alguien intentara falsificar el mensaje, el sistema lo detecta, lanza un error y registra un fallo en el log de auditoría.

```
if id_cp_sesion in CPS_IDX:
    print(f"[DEBUG] CP conocido -> enviando REGISTRADO")
    conn.send("REGISTRADO".encode(FORMAT).ljust(HEADER))
    clave_aes = obtener_clave_registry(id_cp_sesion)
    print(f"[DEBUG] Clave del Registry: {clave_aes is not None}")
    if clave_aes:
        CP_KEYS[id_cp_sesion] = clave_aes
        CP_SOCKETS[id_cp_sesion] = conn
        fernet_cp = Fernet(clave_aes.encode() if isinstance(clave_aes, str) else clave_aes)
        audit("RECONEXION", f"CP {id_cp_sesion} reconectado", ip_origen)
    else:
        print(f"[DEBUG] CP desconocido -> enviando DESCONOCIDO")
        conn.send("DESCONOCIDO".encode(FORMAT).ljust(HEADER))
        clave_aes = obtener_clave_registry(id_cp_sesion)
        print(f"[DEBUG] Clave del Registry: {clave_aes is not None}")
        if clave_aes:
            fernet_cp = Fernet(clave_aes.encode() if isinstance(clave_aes, str) else clave_aes)
        else:
            print(f"[DEBUG] AVISO: sin clave AES, no se podra descifrar el siguiente mensaje")
```

```
# PASO 2: mensajes cifrados
while True:
    msg_length = conn.recv(HEADER).decode(FORMAT).strip()
    print(f"[DEBUG] Header paso2: '{msg_length}'")
    if not msg_length:
        continue

    raw_bytes = conn.recv(int(msg_length))
    print(f"[DEBUG] Bytes recibidos: {len(raw_bytes)}, fernet: {fernet_cp is not None}")

    if fernet_cp:
        try:
            msg = fernet_cp.decrypt(raw_bytes).decode(FORMAT)
            print(f"[DEBUG] Descifrado OK: '{msg[:80]}'")
        except Exception as e:
            print(f"[DEBUG] ERROR descifrado: {e}")
            audit("ERROR_DESCIFRADO", f"No se pudo descifrar mensaje de CP {id_cp_sesion}", ip_origen)
            break
    else:
        msg = raw_bytes.decode(FORMAT)
        print(f"[DEBUG] Mensaje en claro: '{msg[:80]}'")

    if msg == FIN:
        break
```

Por Kafka (Con el Engine): Tiene dos tareas en las funciones de Kafka:
Al enviar (`replyToEngine`): Busca la clave del motor específico, cifra la orden con Fernet y la publica en la cola.

```
def replyToEngine(producer, petition, cp_id, driver_id):
    topic_resp = f"central-to-engine-{cp_id}"
    payload = f"central|{petition}|{cp_id}|{driver_id}"

    # Ciframos el mensaje hacia el Engine usando su clave AES
    clave = CP_KEYS.get(cp_id)
    if clave:
        try:
            f = Fernet(clave.encode() if isinstance(clave, str) else clave)
            payload_cifrado = f.encrypt(payload.encode(FORMAT))
            producer.send(topic_resp, payload_cifrado)
            producer.flush(1)
            print(f"[CENTRAL → ENGINE {cp_id}] Petición cifrada enviada.")
        except Exception as e:
            print(f"[ERROR] No se pudo cifrar hacia ENGINE {cp_id}: {e}")
    else:
        print(f"[AVISO] El CP {cp_id} no tiene clave registrada. No se puede enviar.")
```

Al recibir (receive_messages): Cuando le llega un mensaje del tópico engine-to-central, el contenido está totalmente oculto y la Central no sabe qué motor le está hablando. Para resolverlo, el código ejecuta un bucle que prueba una a una las claves de los CPs activos hasta que encuentra la correcta que logra descifrar el mensaje con éxito.

```
# CANAL ENGINE (CIFRADO FERNET)
# =====
elif origen_topic == TOPIC_ETC:
    descifrado_exitoso = False
    text = ""

    # Como el Engine cifra todo el payload, intentamos descifrar
    # probando las claves activas hasta que una funcione
    for cp_key_id, aes_key in CP_KEYS.items():
        try:
            f = Fernet(aes_key.encode() if isinstance(aes_key, str) else aes_key)
            text = f.decrypt(raw_bytes).decode(FORMAT)
            descifrado_exitoso = True
            break
        except Exception:
            continue

    if descifrado_exitoso:
        parts = text.split("|")
        if len(parts) >= 4 and parts[0].lower() == "engine":
            petition = parts[1]
            cp_id = parts[2]
            driver_id = parts[3]

            print(f"\n[CENTRAL-KAFKA] Recibido ENGINE (Descifrado): {text}")
            attendToEngine(producer, petition, cp_id, driver_id)
    else:
        print("\n[CENTRAL-KAFKA] AVISO: Recibido paquete ENGINE cifrado que no pudo ser desencriptado.")
```

3. EV_W

El módulo EV_W es un servicio independiente cuya función es vigilar el estado meteorológico de las ciudades donde hay Puntos de Carga (CPs). Siguiendo los principios de las arquitecturas orientadas a servicios (SOA),

este módulo no comparte base de datos con la Central, sino que toda la información se intercambia mediante peticiones API REST (HTTP).

Para explicar cómo funciona en tu código, debes dividir su comportamiento en tres tareas clave:

- Recibir nuevas ciudades desde la Central (Su propia API)
Dónde está en el código: En el endpoint `@app.route("/ciudad", methods=["POST"])` dentro de `EV_W.py`.

```
@app_evw.route("/ciudad", methods=["POST"])
def recibir_ciudad():
    data = flask_request.json
    ciudad_raw = data.get("ciudad", "").strip() if data else ""
    if not ciudad_raw:
        return jsonify({"error": "ciudad no especificada"}), 400

    ciudad = ciudad_raw.title()
    if ciudad.lower() == "desconocida":
        return jsonify({"error": "ciudad no válida"}), 400

    if ciudad not in LOCATIONS:
        LOCATIONS[ciudad] = {"estado": "OK", "temperatura": None}
        print(f"[EV_W] Nueva ciudad recibida de Central: {ciudad}")
        return jsonify({"resultado": "OK", "ciudad": ciudad}), 200
    else:
        return jsonify({"resultado": "YA_EXISTE", "ciudad": ciudad}), 200
```

EV_W levanta un servidor web con Flask (por defecto en el puerto 6060). Expone este endpoint para que, cada vez que un Punto de Carga se conecta en una nueva localización, la Central le envíe un mensaje POST con el nombre de la ciudad. Al recibirlo, EV_W añade esa ciudad a su lista de vigilancia interna (LOCATIONS).

- Consultar el clima real (Consumo de API Externa)
En la función `consultar_tiempo(ciudad)`.

```
def consultar_tiempo(ciudad):
    try:
        url = (
            "https://api.openweathermap.org/data/2.5/weather"
            f"?q={ciudad}&appid={OPENWEATHER_API_KEY}&units={UNITS}"
        )
        response = requests.get(url, timeout=5)
        data = response.json()

        if "main" not in data:
            print(f"[EV_W][ERROR] No se pudo consultar {ciudad}: {data.get('message', 'sin datos')}")
            return "ERROR", None

        temp = data["main"]["temp"]
        estado = "KO" if temp < 0 else "OK"
        return estado, temp

    except requests.exceptions.ConnectionError:
        print(f"[EV_W][ERROR] Sin conexión a OpenWeather al consultar {ciudad}")
        return "ERROR", None
    except Exception as e:
        print(f"[EV_W][ERROR] Error inesperado consultando {ciudad}: {e}")
        return "ERROR", None
```

EV_W no se inventa el clima. Utiliza una herramienta profesional externa llamada OpenWeather API. Mediante una petición HTTP GET y una clave segura (API_KEY), el código pregunta por la temperatura exacta de la ciudad. Si la temperatura es menor o igual a 0°C, el sistema salta una alerta y cambia el estado de la ciudad a KO. Si hace buen tiempo, el estado es OK.

- Notificar las alertas a la Central (Consumo de la API de Central)
En la función notificar_central(ciudad, estado, temperatura) y el hilo secundario bucle_monitoreo().

```
def notificar_central(ciudad, estado, temperatura):
    payload = {
        "ciudad": ciudad,
        "temperatura": temperatura
    }
    try:
        response = requests.put(API_CENTRAL_URL, json=payload, timeout=3)
        if response.status_code == 200:
            print(f"[EV_W] Notificado a Central: {ciudad} -> {estado} ({temperatura}°C)")
        else:
            print(f"[EV_W][ERROR] Central respondió {response.status_code} al notificar {ciudad}")
    except requests.exceptions.ConnectionError:
        print(f"[EV_W][ERROR] Imposible conectar con Central. ¿Está arrancada?")
    except Exception as e:
        print(f"[EV_W][ERROR] Error notificando a Central: {e}")
```



```
def ciclo_clima():
    primer_ciclo = True
    while RUNNING:
        for ciudad in list(LOCATIONS.keys()):
            estado_nuevo, temp = consultar_tiempo(ciudad)

            # Solo actuar si obtuvimos datos válidos
            if estado_nuevo == "ERROR":
                continue

            estado_anterior = LOCATIONS[ciudad]["estado"]
            temp_anterior = LOCATIONS[ciudad]["temperatura"]

            LOCATIONS[ciudad]["temperatura"] = temp
            LOCATIONS[ciudad]["estado"] = estado_nuevo

            # Guardar en BD solo si algo cambió
            if estado_nuevo != estado_anterior or temp != temp_anterior:
                guardar_clima(ciudad, estado_nuevo, temp)

            # Notificar a Central: siempre en el primer ciclo, o si cambia el estado
            if primer_ciclo or estado_nuevo != estado_anterior:
                if not primer_ciclo:
                    print(f"[EV_W] Cambio de estado en {ciudad}: {estado_anterior} -> {estado_nuevo} ({temp}°C)")
                    notificar_central(ciudad, estado_nuevo, temp)

        primer_ciclo = False
        time.sleep(CHECK_INTERVAL)
```

EV_W tiene un bucle automático que revisa el tiempo cada pocos segundos. Si detecta que el estado de una ciudad ha cambiado (por ejemplo, de OK a KO por heladas), consume la API de la Central haciendo una petición `requests.post()` al endpoint `/clima` de la Central. Le envía un JSON con la ciudad y el estado para que la Central desactive inmediatamente los cargadores afectados.

4. API_Central

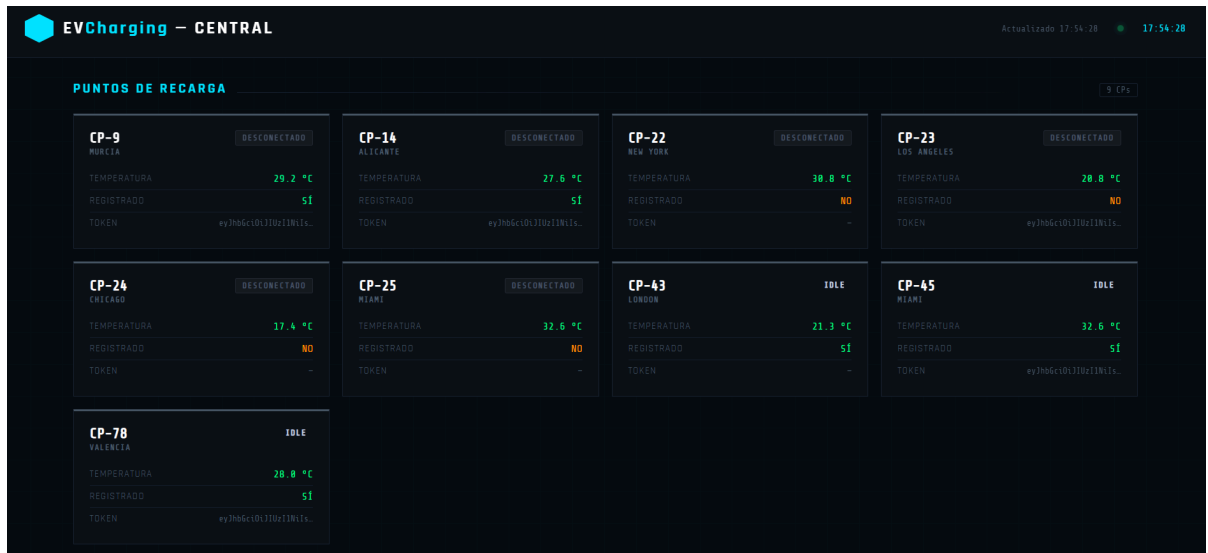
API_Central funciona como un componente de tipo Backend-for-Frontend (BFF) o API de pasarela. Su objetivo es independizar la interfaz web (Frontend) de la lógica interna de la Central (EV_Central). Corre en un puerto separado (por ejemplo, el 8080) y se encarga de servir la aplicación web y redirigir de forma segura las peticiones hacia el núcleo del sistema.

- Servidor de archivos estáticos para la interfaz gráfica.

```
# =====
# FRONTEND
# =====

@app.route("/")
def index():
    return send_from_directory(os.path.join(BASE_DIR, "static"), "index.html")

@app.route("/ping")
def ping():
    return "pong"
```



Este componente es el encargado de levantar el entorno visual de la aplicación web de gestión. Al configurar Flask con el parámetro `static_folder`, definimos la ruta del Frontend. Cuando el administrador del sistema accede a la raíz del servidor (`@app.route("/")`), la API intercepta la petición y responde con el archivo `index.html`. A partir de ahí, la página web puede realizar peticiones locales para pintar el mapa y renderizar de forma visual e intuitiva la red de carga.

- Mecanismo de delegación y traducción de peticiones.

```
@app.route("/clima", methods=["PUT"])
def recibir_clima():
    """
    Recibe notificación de EV_W y la delega a EV_Central para que
    sea ella quien actualice la BD y gestione el estado de los CPs.
    Espera JSON: { "ciudad": "Barrow", "temperatura": -6.99 }
    """
    data = request.json
    if not data or "ciudad" not in data or "temperatura" not in data:
        return jsonify({"error": "Faltan campos: ciudad, temperatura"}), 400

    print(f"[API_Central] Notificación de clima recibida: {data} - delegando a EV_Central")

    try:
        # Adaptamos el formato al que espera EV_Central: { "localizacion": ..., "estado": ... }
        ciudad = data["ciudad"]
        temperatura = data["temperatura"]
        estado_clima = "KO" if temperatura <= 0 else "OK"

        resp = requests.post(
            f"{CENTRAL_URL}/clima",
            json={
                "localizacion": ciudad,
                "estado": estado_clima,
                "temperatura": temperatura
            },
            timeout=5
        )
        return jsonify({
```

API_Central actúa como un proxy inverso inteligente. Un ejemplo claro está en el endpoint que gestiona las notificaciones de clima. La API

recibe los datos directamente en un formato simplificado ({ "ciudad": ..., "temperatura": ... }). El código procesa esta entrada, calcula si el clima debe traducirse a un estado crítico (KO si es menor o igual a cero), reestructura el payload JSON al formato exacto que exige la Central original y realiza una petición interna `requests.post(f"{CENTRAL_URL}/clima", json=...)`. De esta manera, el emisor externo nunca sabe dónde está ni cómo procesa la información el backend principal; todo se centraliza y traduce a través de esta API.

- Control de concurrencia y lectura segura mediante exclusión mutua

```
def cargar_db():
    if not os.path.exists(DB_FILE):
        return {"clientes": [], "cps": [], "climas": []}
    with open(DB_FILE, "r", encoding="utf-8") as f:
        return json.load(f)

def buscar_cp(cps, id_cp):
    return next((cp for cp in cps if cp["id"] == id_cp), None)
```

Dado que el Frontend solicita constantemente actualizaciones del estado de los Puntos de Carga, clientes y alertas, la API necesita acceder al archivo persistente `db.json`. Para evitar condiciones de carrera (es decir, que el hilo de la API intente leer el fichero JSON exactamente al mismo milisegundo en que el proceso principal de `EV_Central` está escribiendo un nuevo estado por un mensaje de Sockets o Kafka), hemos implementado un mecanismo de exclusión mutua mediante `lock = threading.Lock()`. En la función `cargar_db()`, el hilo adquiere el cerrojo antes de abrir el archivo, garantizando que la lectura sea completamente atómica, limpia, consistente y libre de corrupciones de datos.

5. EV_Registry

El módulo `EV_Registry` actúa como la base de confianza y el componente de gobernanza de identidades en nuestra arquitectura distribuida. Su propósito fundamental es autenticar y matricular oficialmente los puntos de carga, proveyendo las credenciales iniciales que la Central exigirá más adelante. Los tres aspectos más importantes de su código e implementación son:

- El proceso autónomo de Registro de CPs (`/register`).

```
@app.put("/register", status_code=201)
def registrar_cp(cp: CPRegistro):
    db = cargar_db()
    if buscar_cp(db, cp.id):
        raise HTTPException(status_code=409, detail="CP ya registrado")

    clave_aes = crear_y_guardar_clave_aes(cp.id)
    token = generar_token(cp.id)

    db["cps"].append({
        "id": cp.id,
        "location": cp.location,
        "registrado": True,
        "estado": "OFFLINE",
        "token": token,
        "aes_key": clave_aes
    })
    guardar_db(db)

    return {
        "status": "registrado",
        "id": cp.id,
        "token": token,
        "aes_key": clave_aes
    }
```

El Registry expone un servicio REST síncrono que incluye el endpoint POST /registro. Cuando un nuevo Punto de Carga físico se despliega, lo primero que hace es invocar esta ruta enviando sus datos básicos en un JSON. El código del Registry valida la solicitud y, para garantizar la seguridad criptográfica desde el primer momento, genera dinámicamente una clave simétrica única para ese CP utilizando `Fernet.generate_key()`. Esta clave se guarda localmente en el servidor (en un fichero o diccionario persistente) asociada al ID del cargador y se le devuelve al CP en la respuesta HTTP. A partir de ese momento, el CP ya está "matriculado" oficialmente en el sistema.

- Emisión de Tokens JWT Firmados.

```
def generar_token(id_cp: str) -> str:
    payload = {
        "id": id_cp,
        "exp": time.time() + 300
    }
    token = jwt.encode(payload, SECRET_KEY, algorithm="HS256")
    if isinstance(token, bytes):
        token = token.decode()
    return token
```

El Registry no solo registra, sino que da fe de la identidad de los componentes en tiempo real. Dispone de un endpoint o mecanismo para generar tokens JWT (JSON Web Tokens) utilizando una clave secreta compartida (SECRET_TOKEN = "evregistry2425"). Cuando un Monitor quiere conectarse a la Central, le solicita un token al Registry. El Registry empaqueta los datos del CP (como su ID y su localización) y los firma digitalmente. Este token JWT actúa como un "pasaporte digital" con tiempo de expiración limitado. El Monitor se lo presentará a la Central y esta, al verificar la firma del Registry, sabrá instantáneamente que el CP es auténtico sin necesidad de volver a preguntar.

- El Endpoint de Consulta Segura para la Central.

```
def obtener_clave_registry(id_cp: str):
    try:
        url = f"http://{REGISTRY_HOST}:{REGISTRY_PORT}/cp/{id_cp}"
        resp = requests.get(url, timeout=5)
        if resp.status_code == 200:
            return resp.json().get("aes_key")
        else:
            audit("REGISTRY_ERROR", f"Registry devolvió {resp.status_code} para CP {id_cp}")
            return None
    except Exception as e:
        audit("REGISTRY_ERROR", f"No se pudo contactar con Registry para CP {id_cp}: {e}")
        return None

@app.get("/cp/{id}")
def consultar_cp(id: str):
    db = cargar_db()
    entrada = buscar_cp(db, id)
    if not entrada:
        raise HTTPException(status_code=404, detail="CP no registrado")
    return entrada
```

El Registry expone una ruta de consulta exclusiva para la Central. Cuando un CP se intenta conectar a la Central mediante Sockets, la Central realiza una petición HTTP GET interna al Registry para

descargar la clave AES que le corresponde a ese ID de CP. El Registry busca en su base de datos local y, si el CP es válido, le transfiere de forma segura la clave a la Central. Gracias a este diseño, las claves simétricas nunca viajan por canales inseguros ni están expuestas públicamente; se gestionan de forma estrictamente privada entre el Registry y la Central.

6. Despliegue del proyecto

```
C:\Users\Carlos\OneDrive\Público\Escritorio\SD>python API_Central.py
```

- Arrancar el API_Central: Se ejecuta en el puerto 8080. Lee los archivos de la carpeta static/ para que podamos abrir el navegador en <http://localhost:8080> y ver el Frontend (el mapa y los cargadores). Además, se conecta internamente a la base de datos db.json con hilos seguros (Lock) para mostrar los datos en tiempo real.

```
C:\Users\Carlos\OneDrive\Público\Escritorio\SD>python EV_Registry.py
```

- Arrancar EV_Registry: Este archivo levanta su propio servidor HTTP (por defecto en el puerto 9100). Se queda a la escucha esperando que los Puntos de Carga se registren y que la Central le pida las claves de cifrado. Si no se arranca primero, la Central fallará al intentar validar los CPs.

```
C:\Users\Carlos\OneDrive\Público\Escritorio\SD>python EV_Central.py
Introduce la IP del servidor central: 162.198.0.25
Introduce la IP del EV_Registry [localhost]:
Introduce la URL de EV_W (ej. http://172.21.42.5:6060) [dejar vacío si no hay]:
```

- Arrancar la Central: Al arrancarlo, el script nos pedirá por teclado tres datos clave:

IP del Servidor Central: (ej. localhost o la IP de la máquina).

IP del EV_Registry: Para saber a dónde conectarse a pedir las llaves AES.

URL de EV_W: La dirección del módulo del clima (ej. <http://localhost:6060>).

Abre el socket TCP en el puerto 5050 para escuchar a los Monitores, activa el consumidor de Kafka para escuchar a los Motores y levanta su Flask interno (puerto 8000) para comunicarse con las APIs.

```
C:\Users\Carlos\OneDrive\Público\Escritorio\SD>python EV_W.py --apikey 44b0f33251e31ec58eac825e9a67604e --central http://127.0.0.1:8080/
```

Arranca el EV_W: Levanta un servidor Flask en el puerto 6060. La Central le irá enviando las ciudades que debe vigilar a través de su endpoint /ciudad. El archivo inicia un bucle asíncrono que hace peticiones GET a OpenWeather y avisa a la Central si hay heladas ($\leq -0\text{ C}^\circ$).

```
C:\Users\Carlos\OneDrive\Público\Escritorio\SD>python ChargingPoint.py --central 192.168.0.25:5050 --engine 192.168.0.25:17000 --id 78 --localizacion valencia --broker 192.168.0.25:9092
```

- Arrancar el Monitor y el Engine: A continuación, se inician el Monitor y el Engine en otro ordenador que comparta la misma red Ethernet que el equipo donde se encuentra la Central. Estos dos componentes se ejecutarán desde el archivo “ChargingPoint.py”. Al archivo se les debe pasar los parámetros necesarios (--central --engine , --id --localizacion --broker) para que puedan conectarse correctamente y empezar a intercambiar mensajes.

```
C:\Users\Carlos\OneDrive\Público\Escritorio\SD>python EV_Driver.py 192.168.0.25:9092 94
```

- Arrancar el Driver: Finalmente, se ejecuta el Driver desde un tercer ordenador, también conectado a la misma red local. Este componente será el encargado de enviar las peticiones a la Central y recibir las respuestas a través de Kafka